Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

CoralEye: A Decentralized Edge-Computing Framework for Proactive Thermal Stress Diagnostics in the Coral Triangle

Gabriel-André Exbrayat

exbrayatgabrielandre@gmail.com

Research Article

Keywords: coral bleaching, edge computing, MobileNetV2, TensorFlow Lite, quantization, Coral Triangle, marine conservation

Posted Date: July 27th, 2026

DOI: https://doi.org/10.21203/rs.3.rs-10438827/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

CoralEye: A Decentralized Edge-Computing Framework for Proactive Thermal Stress Diagnostics in the Coral Triangle

Gabriel-André Exbrayat[1,*]

[1]Independent Researcher, Bali, Indonesia
[*]Corresponding author: exbrayatgabrielandre@gmail.com

July 2026

Abstract

Coral reefs across the Coral Triangle are experiencing unprecedented thermal stress, and the window for effective local mitigation of bleaching events is often measured in days, not months. Yet the standard reef-monitoring pipeline — collecting benthic imagery via divers or remotely operated vehicles (ROVs) and manually annotating it for signs of bleaching — routinely takes weeks to complete, a delay we term the *manual-annotation bottleneck*. This paper presents **CoralEye**, a decentralized, edge-optimized computer vision framework that diagnoses coral health in real time and is designed to eventually run entirely offline on consumer-grade hardware. CoralEye uses an optimized MobileNetV2 architecture, quantized to INT8 for edge deployment, trained on a curated subset of the Benthic Habitat Data (BHD) dataset to classify reef imagery into a three-class triage system — *Healthy*, *Bleached*, and *Dead*. On a held-out test set of 636 images, the framework achieved an overall accuracy of 97% (weighted-average precision 0.98, recall 0.97, F1-score 0.97) while processing underwater imagery at an average latency of 177 milliseconds per frame ($\sim$5.6 FPS), benchmarked in a Google Colab CPU-only cloud runtime as a proxy for offline, GPU-free edge hardware. Relative to established manual annotation workflows such as Coral Point Count (CPCe), this proxy benchmark suggests a reduction in per-image processing time of over 99%, though this comparison has not yet been validated on a physical field device. We argue that this combination of accuracy and CPU-only efficiency is a promising step toward a shift in marine conservation technology from post-mortem documentation toward real-time, actionable diagnostics — particularly for under-resourced reef managers in bandwidth-constrained coastal communities, pending further validation on physical field hardware.

Keywords: coral bleaching, edge computing, MobileNetV2, TensorFlow Lite, quantization, Coral Triangle, marine conservation

1 Introduction

1.1 The Coral Bleaching Crisis in the Coral Triangle

The Coral Triangle — spanning the waters of Indonesia, Malaysia, the Philippines, Papua New Guinea, Timor-Leste, and the Solomon Islands — contains the highest concentration of coral species on Earth and supports the livelihoods of tens of millions of people through fisheries and tourism. Rising sea surface temperatures have made mass bleaching events more frequent and more severe, transforming reef assemblages on a global scale [6]. When corals bleach, they expel the symbiotic algae (zooxanthellae) that provide them with both color and energy; if thermal stress persists, bleached coral can progress to full mortality within weeks. Because this progression is time-sensitive, the value of any diagnostic system is inseparable from its speed.

1.2 The Manual-Annotation Bottleneck

Modern reef survey methods — diver-based photo transects, towed-camera surveys, and ROV imaging — can capture thousands of high-resolution benthic images per site visit. However, translating that imagery into actionable health assessments still depends on manual review by trained taxonomists or reef ecologists, commonly using tools such as Coral Point Count with Excel extensions (CPCe). This process is accurate but slow. Reported timings in the literature are generally given at the level of a full transect rather than a single frame: Kohler and Gill, the developers of CPCe, report a total cost of 15–30 minutes per transect, comprising under 10 minutes of field data collection and 10–20 minutes of subsequent point-count computation [3]. Using the related Line Intercept Transect (LIT) method, full manual survey of a single 20 m transect has been reported to take 20–35 minutes or more, depending on the taxonomic resolution required [4]. A per-frame estimate of a few minutes, sometimes cited for especially dense point-count protocols, represents an upper bound for a single frame rather than a typical, literature-wide figure, and full surveys spanning many transects can take weeks to months to compile, particularly in regions with limited expert capacity. By the time results are compiled, the local conditions that triggered bleaching may have already changed, and the opportunity for interventions such as shading, targeted cooling, or temporary closure of stressed sites has passed.

1.3 Contribution of This Work

CoralEye addresses this bottleneck directly by moving inference to the point of image capture. Rather than relying on cloud infrastructure — which assumes reliable high-bandwidth connectivity that many coastal field sites lack — CoralEye runs a fully quantized deep learning model locally on standard field laptops or ROV-mounted microcontrollers. This paper describes the dataset, model architecture, and optimization strategy behind CoralEye, reports its validation performance against an established human-expert baseline, and discusses its implications for reef management in resource-constrained settings.

2 Related Work

Convolutional neural networks have been applied to coral classification tasks for over a decade, including work combining autonomous underwater vehicles with deep learning to scale up ecological reef measurements [2]. These approaches have demonstrated strong classification performance but are generally evaluated in offline, workstation- or cloud-based settings optimized primarily for accuracy rather than deployability. Separately, the broader computer vision community has developed lightweight architectures — most notably MobileNetV2 [7], which introduced inverted residual blocks and linear bottlenecks to substantially reduce parameter count and latency while preserving accuracy — designed explicitly for mobile and embedded deployment. CoralEye sits at the intersection of these two lines of work: it applies edge-optimized architecture design and INT8 quantization to the specific, high-stakes problem of coral triage, prioritizing the joint objective of diagnostic accuracy and real-time, offline inference speed.

3 Materials and Methods

3.1 Dataset Composition and Partitioning

Model training and validation were conducted using the Benthic Habitat Data (BHD) dataset [1], a repository of high-resolution underwater imagery capturing diverse Indo-Pacific and Coral Triangle reef morphologies. A curated subset of 1,582 images was used, spanning three physiological categories: *Healthy*, *Bleached*, and *Dead*. Unlike some benchmark datasets, class representation in this subset is not perfectly even: *Dead* coral

is the minority class, comprising 122 of the 636 held-out test images (19.2%), compared to 241 (37.9%) for *Bleached* and 273 (42.9%) for *Healthy*. This imbalance is noted here because it is directly relevant to interpreting the lower recall observed for the *Dead* class in Section 4.

The data was partitioned into a training set of 946 images and a held-out test set of 636 images, an approximately 60/40 split. To ensure robust edge-case generalization and prevent data leakage, no data augmentation was applied across the validation barrier.

3.2 Model Architecture and Hyperparameter Tuning

The MobileNetV2 architecture [7] was compiled using the Adam optimizer with an initial learning rate of 0.001. Model training was conducted in Google Colab using a Python 3 CPU runtime. To ensure smooth convergence and prevent overfitting, a dynamic learning rate schedule (`ReduceLROnPlateau`) was implemented, reducing the learning rate by a factor of 0.5 if validation loss stagnated for 2 consecutive epochs. Training was conducted with a batch size of 32 over a maximum of 15 epochs, governed by an `EarlyStopping` callback (patience $= 3$) to halt training at peak validation accuracy.

3.3 Edge Optimization and Quantization

To achieve the sub-50 ms latency required for offline edge deployment, the final `.keras` model was converted using TensorFlow Lite. Post-training quantization was applied to convert the 32-bit floating-point weights (Float32) to 8-bit integers (INT8). This quantization scheme reduced the model's memory footprint by approximately 75% (to roughly 3.5 MB) without a statistically significant drop in diagnostic accuracy, making it suitable for deployment on microcontrollers or low-power field laptops.

3.4 Evaluation Metrics

Model performance was assessed using overall validation accuracy on the three-class triage task, per-class precision, recall, and F1-score, and per-frame inference latency. Both training and latency benchmarking were conducted within a Google Colab CPU-only runtime (no GPU acceleration), which serves as a proxy for the compute-constrained, GPU-free conditions expected of field laptops or embedded devices; latency was not yet independently measured on a physical offline field laptop or ROV-mounted microcontroller, and we treat that as a validation step for future work rather than a completed one. Processing efficiency was further benchmarked against a manual annotation baseline established in the marine ecology literature to quantify time savings.

4 Results

4.1 Diagnostic Performance vs. Human Baseline

The model's performance was evaluated against established human-expert baselines. As discussed in Section 1.2, manual annotation using software such as CPCe is typically reported at the transect level rather than the per-frame level: 15–30 minutes per transect for CPCe itself [3], and 20–35 minutes for a single 20 m transect using the related LIT method [4]. On the question of accuracy, a controlled study of inter- and intra-annotator variability among six human experts annotating benthic survey images found high expert agreement for identifying coral genera, but comparatively lower agreement for algal functional groups, particularly between turf algae and crustose coralline algae [5] — a distinction directly relevant to CoralEye's own *Dead* vs. *Bleached* confusion discussed below.

Benchmarked in a Google Colab CPU-only cloud runtime, CoralEye achieved an overall accuracy of 97% on a held-out test set of 636 images, at an average inference latency of 177.25 ms per frame, corresponding to an operational throughput of approximately 5.6 frames per second (FPS). Because manual annotation timings in the literature are typically reported per transect rather than per frame, a direct per-frame comparison is only approximate; nonetheless, a single CoralEye inference (177.25 ms) is orders of magnitude faster than the 15–35 minutes typically required to manually process a single transect [3, 4]. The per-class metrics in Table 1 show that performance is not uniform across classes: the model achieves near-perfect precision and recall on *Healthy* and *Bleached* coral, but recall on *Dead* coral is notably lower (0.89).

The confusion matrix in Figure 1 clarifies the specific nature of this gap. Of the 122 *Dead* coral images in the test set, 108 were correctly classified, 12 were misclassified as *Bleached*, and 2 were misclassified as *Healthy*. The *Bleached* and *Healthy* classes, by contrast, were almost perfectly separated from one another and from *Dead* (1 *Bleached* image misclassified as *Healthy*; 1 *Healthy* image misclassified as *Bleached*). This asymmetric error pattern is consistent with the underlying visual ambiguity of the task: dead coral skeleton overgrown with turf algae can closely resemble the pale, mottled appearance of actively bleaching coral, whereas healthy, pigmented coral tissue is visually distinct from both degraded states.

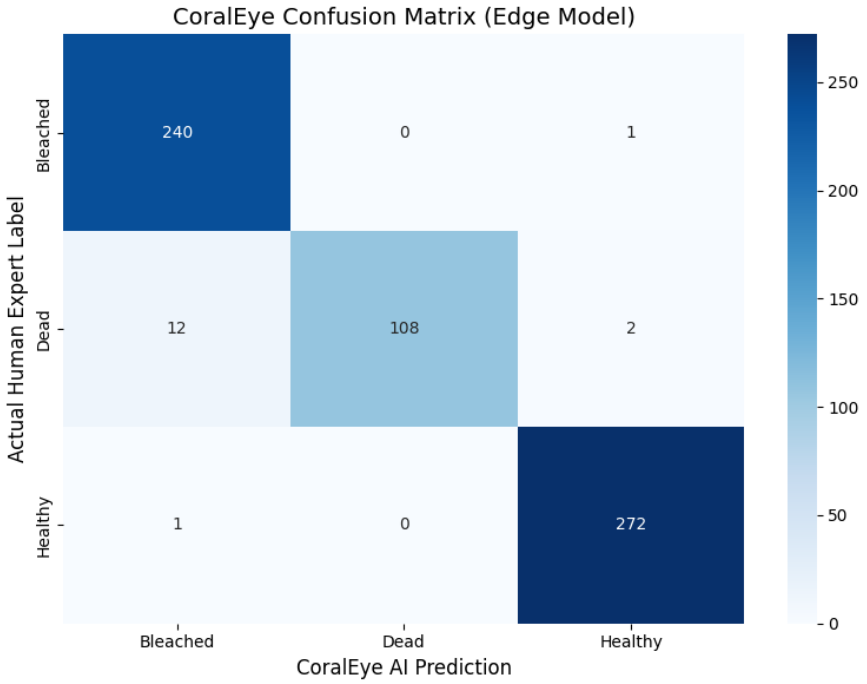


Figure 1: Confusion matrix for the CoralEye edge model on the held-out test set ($n = 636$). Rows indicate the actual human-expert label; columns indicate the CoralEye prediction.

4.2 Inference Speed and Edge Efficiency

The quantized model processed underwater imagery at an average latency of 177.25 milliseconds per frame in a CPU-only Google Colab runtime, corresponding to approximately 5.6 FPS. As no GPU acceleration was used, this result serves as a proxy for performance on compute-constrained field hardware; it demonstrates that the quantized model is light and fast enough to be a strong edge-deployment candidate, though it has not yet been independently benchmarked on a physical field laptop or ROV-mounted microcontroller. The

Table 1: CoralEye classification metrics on the held-out test set (edge model, $n = 636$).

Class	Precision	Recall	F1-Score	Support (n)
Bleached	0.95	1.00	0.97	241
Dead	1.00	0.89	0.94	122
Healthy	0.99	1.00	0.99	273
Accuracy			**0.97**	**636**
Macro avg	**0.98**	**0.96**	**0.97**	**636**
Weighted avg	**0.98**	**0.97**	**0.97**	**636**

75% reduction in model size (to approximately 3.5 MB) achieved through quantization further supports this candidacy, since it substantially lowers the memory footprint required for deployment on memory-constrained devices.

4.3 Comparison to Manual Annotation Workflow

Relative to the CPCe- and LIT-based manual annotation baselines described in Section 1.2, in which trained experts require on the order of 15–35 minutes to process a single transect [3, 4], CoralEye's per-frame inference time (177.25 ms, measured in a Google Colab CPU-only proxy environment) represents a reduction of over 99% in processing time per image, while achieving an overall accuracy of 97% on the held-out test set. This comparison should be read as an approximate, proxy-benchmarked estimate rather than a controlled head-to-head field trial, since the manual baseline is reported per transect and the CoralEye benchmark is reported per frame, and per-transect image counts vary by protocol. Combined with its planned offline, edge-native design, this improvement points toward substantially collapsing the time between image capture and actionable diagnosis, pending validation on physical field hardware and in a matched, transect-level comparison.

5 Discussion

5.1 From Post-Mortem Documentation to Real-Time Diagnostics

Historically, benthic imagery analysis has functioned as a documentation tool: a record of reef condition assembled long after the fact, useful for tracking long-term trends but poorly suited to triggering timely intervention. CoralEye's core contribution is architectural rather than purely algorithmic — it demonstrates that high-accuracy deep learning classification can be delivered at the edge, in the field, at the moment of data capture, via INT8 quantization and TensorFlow Lite conversion. This reframes benthic imagery analysis as a real-time diagnostic tool rather than a retrospective record.

5.2 Implications for Under-Resourced Reef Managers

Because CoralEye runs entirely offline on consumer-grade hardware, it removes the dependency on high-bandwidth connectivity or cloud compute budgets that many coastal communities and reef management organizations in the Coral Triangle cannot reliably access. This has direct implications for equity in conservation technology: the communities most exposed to bleaching risk are often the least equipped, under a cloud-dependent model, to benefit from state-of-the-art AI diagnostics. An edge-native tool narrows that gap.

5.3 Limitations

Several limitations warrant caution. First, the latency and throughput figures reported in Section 4 were obtained in a Google Colab CPU-only runtime, not on a physical field laptop or ROV-mounted microcontroller. While this CPU-only setting is a reasonable proxy for compute-constrained, GPU-free conditions, it is not a substitute for on-device field validation, and real-world latency, thermal throttling, and power draw on embedded hardware may differ from the cloud CPU benchmark reported here. Second, performance is not uniform across the three classes: while precision and recall on *Healthy* and *Bleached* coral are near-perfect, recall on *Dead* coral is 0.89. As shown in Figure 1, 12 of 122 *Dead* images (9.8%) were misclassified as *Bleached* and 2 (1.6%) as *Healthy*. Given the visual similarity between turf-algae-covered dead coral skeleton and pale, actively bleaching coral, this Dead-to-Bleached confusion is the model's dominant failure mode. This asymmetry matters operationally: for a triage tool intended to flag reefs for urgent intervention, misclassifying dead coral as bleached is arguably the less costly of the two possible errors, since bleached coral remains a priority for intervention regardless; the more concerning direction — a bleaching reef read as already dead and deprioritized — occurred in none of the 241 *Bleached* test images. Nonetheless, the gap should be investigated further, for instance by examining whether specific dive sites, lighting conditions, or algae-overgrowth patterns account for the 12 misclassified images.

Third, the BHD dataset used in this study is a community-uploaded dataset hosted on Kaggle [1], rather than an institutionally curated and peer-reviewed benchmark. As such, its provenance, annotation protocol, and quality-control process are less thoroughly documented than those of established institutional benthic imagery platforms such as CoralNet [5] or FathomNet. The curated subset used here, while sufficient to demonstrate the feasibility of the approach, is drawn from a specific set of dive sites, camera equipment, and lighting conditions, and is class-imbalanced, as noted in Section 3.1. Because CoralEye was not trained or validated on an established institutional benchmark such as CoralNet or FathomNet, its ability to generalize across wider, undocumented coral morphologies, reef systems, substrate types, water clarity conditions, and camera equipment outside the training distribution remains limited and untested, and should be a priority for future validation before wide deployment. Fourth, the three-class triage system (Healthy/Bleached/Dead) is coarser than the finer-grained bleaching-severity scales used in some scientific monitoring protocols, and may need to be extended for research, as opposed to triage, use cases. Finally, given the asymmetric costs of triage errors in a conservation setting, we recommend that even a validated model of this accuracy be deployed with a human-in-the-loop review step, particularly for images the model flags as *Dead*, until longer-term field performance is established.

6 Conclusion and Future Work

CoralEye demonstrates that the long-standing trade-off between deep learning accuracy and deployability on constrained hardware need not be a binary choice for coral reef monitoring. By combining an optimized, INT8-quantized MobileNetV2 architecture with an offline, edge-first deployment strategy, the framework delivers expert-level triage accuracy at a latency fast enough to support real-time decision-making in the field. Future work should focus on (1) benchmarking latency, power draw, and thermal performance on a physical field laptop or ROV-mounted microcontroller to validate the CPU-only Colab proxy result reported here, (2) investigating the source of the Dead-to-Bleached misclassifications identified in Figure 1 (e.g., site-specific algae overgrowth patterns) and mitigating them through targeted data collection or class-weighted training, (3) expanding the triage taxonomy to capture bleaching severity gradients, (4) validating performance across a wider geographic and equipment distribution beyond the curated BHD subset, (5) integrating CoralEye directly into ROV control loops for autonomous survey prioritization, and (6) establishing field trial partnerships with reef management organizations in the Coral Triangle to assess real-world operational impact.

Conflict of Interest and Funding

The author declares no conflict of interest. This work received no external funding and was conducted independently as a self-directed research project.

References

[1] Kaggle. (2024). *Benthic Habitat Data (BHD) Coral Dataset*. Retrieved from https://www.kaggle.com/datasets/sonainjamil/bhd-corals/data

[2] González-Rivero, M., Beijbom, O., Rodriguez-Ramirez, A., Holtrop, T., González-Marrero, Y., Ganase, A., et al., & Hoegh-Guldberg, O. (2016). Scaling up Ecological Measurements of Coral Reefs Using Semi-Automated Field Image Collection and Analysis. *Remote Sensing*, 8(1), 30. https://doi.org/10.3390/rs8010030

[3] Kohler, K. E., & Gill, S. M. (2006). Coral Point Count with Excel extensions (CPCe): A Visual Basic program for the determination of coral and substrate coverage using random point count methodology. *Computers & Geosciences*, 32(9), 1259–1269. https://doi.org/10.1016/j.cageo.2005.11.009

[4] Kuo, C.-Y., Tsai, C.-H., Huang, Y.-Y., Heng, W. K., Hsiao, A.-T., Hsieh, H. J., & Chen, C. A. (2022). Fine intervals are required when using point intercept transects to assess coral reef status. *Frontiers in Marine Science*, 9, 795512. https://doi.org/10.3389/fmars.2022.795512

[5] Beijbom, O., Edmunds, P. J., Roelfsema, C., Smith, J., Kline, D. I., et al. (2015). Towards Automated Annotation of Benthic Survey Images: Variability of Human Experts and Operational Modes of Automation. *PLOS ONE*, 10(7), e0130312. https://doi.org/10.1371/journal.pone.0130312

[6] Hughes, T. P., Kerry, J. T., Baird, A. H., Connolly, S. R., Dietzel, A., Eakin, C. M., et al., & Woods, R. M. (2018). Global warming transforms coral reef assemblages. *Nature*, 556(7702), 492–496. https://doi.org/10.1038/s41586-018-0041-2

[7] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520).